Assignment-3

* Problem Statement

- Write a C program to create a student database using an array of structure, Apply searching & sorting techniques.

* Objective:

i) To understand use of an array of structures for maintaining records.

ii) To implement & analyze searching techniques.

iii) To implement & analyze sorting techniques.

iv) To compare insertion sort, Selection Sort, Shell sort.

* Theory

- Searching & sorting are two of most fundamental operation in data structures. They are widely used in computer application such as database management system.

- Linear search is the simplest searching technique It is also known as Sequential Search.

- Binary Search: It is efficient searching algorithm that works only on sorted data.

- Types of sorting:

i) Bubble Sort
ii) Selection Sort
iii) Insertion sort
iv) Shell sort
v) Merge sort
vi) Quick sort.

* Pseudo code

1) Linear Search

```
Start
for i ← 0 to n-1
    if A[i] = key then
        print element found
    End if
End for Else
    print Not found.
End for
Stop
```

2. Binary Search

```
Start
low ← 0
high ← n-1
while low ≤ high do
        mid ← (low/high) /2
If A[mid] = key then
        print element found
Stop
Else if key < A[mid] then
        high ← mid - 1
Else
   low ← mid + 1
End if
End for
Stop.
```

3. Algorith Insertion

```
Start
for i ← 1 to n - 1 do
      key ← A[i]
      j = i - 1
      while j ≥ 0 && A[j] > key do
          A[j+1] = A[j]
          j = j - 1
        End while
      A[j+1] ← key
      End for
      print Array
      Stop.
```

4. Algorithm Selection_Sort

Start
For i ← 0 to n-2 do
min = i
for j ← i+1 to n-1 do
if A[j] < A[min] then
min = j
End if
End for
Swap A[i] & A[min]
End for
Print sorted array
Stop.

5. Algorithm Shell_Short

Start
gap = n/2
while gap => 0 do
for i = gap to n-1 do
temp = A[i]
j = i
while j >= gap && A[j-gap]
A[j] = A[j-gap]
j = j-gap
End while
A[j] ← Temp
End for
End while
print array
Stop.

- Time complexity:

	Best	Avg	Worst
i) Linear Search	O(1)	O(n)	O(n)
ii) Binary Search	O(1)	O(logn)	O(logn)
iii) Insertion Search	O(n)	O(n²)	O(n²)
iv) Selection Sort	O(n²)	O(n²)	O(n²)
v) Shell Sort	O(nlogn)	Depends on gap	O(n²)

* Conclusion:

They, implemented different Searching & sorting methods on the student db.

This system is able to perform sorting under different cases.

* FAQs:

Q 1. What is the meaning of database? How to maintain in C?

A- A database is an organized collection of data that is stored & managed so that it can be easily accessed, modified, searched & update.

- C does not have built in database system like MySQL or Oracle.
- We can maintain data using:
  i) Structure
  ii) Arrays
  iii) File Handling
  iv) Function.

Q2 What are the applications of Searching?

A- Searching is the process of finding a particular element or record from a collection of data.

i) Database Management: Finding a particular student employee, customer, etc.

ii) File Systems: Searching for specific file or folder.

iii) Information Retrieval.

iv) Banking Systems: For customer accounts & transaction.

v) Ecommerce - To find a product, service, etc.

vi) Hospital Management.

Q3. Compare Linear Search & Binary Search.

Basis	Linear Search	Binary Search
Method	Checks element one by one	Divides the array into halves.
Data Requirement	Array need not be sorted	Array must be sorted.
Best Case	O(1)	O(1)
Speed	Slower	Faster

Q4. What are the application of sorting?

i) Searching : Sorting makes easy to search elements.

ii) Manage Records : In database, sorted data helps to fetch data faster.

iii) Data Analysis : It makes easy to analyze a data set.

iv) Ranking System

v) File Management : Arranging files acc. to name id, etc.

Q5. Compare & contrast: Selection, Insertion & Shell sort

Basis	Selection	Insertion	Shell
Best case	$O(n^2)$	$O(n)$	depend on gap
Stability	No	Yes	No
Speed	Slow	Fast	Medium
Working	Selects smallest elements & places correct position	Takes one elemt & insert it into the sorted part	Perform insertion sort on an element & seperate by gap.

Q6. What is external & Internal sorting, inplace, stable features of sorting.

i) Internal Sorting: In these, all the data to be sorted fits into the main memory that is the RAM.

ii) External Sorting: It is used when the data is too large to fit into main memory. It uses secondary storage.

iii) Inplace Sorting: It sorts data using very little extra memory, usually O(1) auxiliary space.

31/7

C Program:

```
#include <stdio.h>

struct Student
{
    int rollNo;
    char name[50];
    float marks;
};

void inputStudents(struct Student s[], int n)
{
    int i;

    for (i = 0; i < n; i++)
    {
        printf("\nEnter details of student %d\n", i + 1);

        printf("Roll No.: ");
        scanf("%d", &s[i].rollNo);

        printf("Name: ");
        scanf(" %[^\n]", s[i].name);

        printf("Marks: ");
        scanf("%f", &s[i].marks);
    }
}

void displayStudents(struct Student s[], int n)
{
    int i;

    printf("\nRoll No.\tName\t\tMarks\n");
    printf("----------------------------------------\n");

    for (i = 0; i < n; i++)
    {
        printf("%d\t\t%s\t\t%.2f\n",
               s[i].rollNo, s[i].name, s[i].marks);
    }
}

void searchStudent(struct Student s[], int n)
{
    int roll, i, found = 0;

    printf("\nEnter roll number to search: ");
    scanf("%d", &roll);

    for (i = 0; i < n; i++)
    {
        if (s[i].rollNo == roll)
        {
```

```
            printf("\nStudent Found\n");
            printf("Roll No. : %d\n", s[i].rollNo);
            printf("Name     : %s\n", s[i].name);
            printf("Marks    : %.2f\n", s[i].marks);
            found = 1;
            break;
        }
    }

    if (found == 0)
        printf("\nStudent not found.\n");
}

void sortStudents(struct Student s[], int n)
{
    int i, j;
    struct Student temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (s[j].rollNo > s[j + 1].rollNo)
            {
                temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }
        }
    }

    printf("\nStudents sorted by roll number.\n");
}

int main()
{
    int n;

    printf("Enter number of students: ");
    scanf("%d", &n);

    struct Student students[n];

    inputStudents(students, n);

    printf("\nStudent Database:\n");
    displayStudents(students, n);

    searchStudent(students, n);

    sortStudents(students, n);

    printf("\nSorted Student Database:\n");
    displayStudents(students, n);
```

```
    return 0;
}
```

Output:

```
Enter number of students: 4

Enter details of student 1
Roll No.: 1
Name: Eshan
Marks: 92

Enter details of student 2
Roll No.: 2
Name: Archit
Marks: 94

Enter details of student 3
Roll No.: 3
Name: Aarushi
Marks: 97

Enter details of student 4
Roll No.: 4
Name: Anushka
Marks: 97

Student Database:

Roll No.        Name            Marks
---------------------------------------
1               Eshan           92.00
2               Archit          94.00
3               Aarushi         97.00
4               Anushka         97.00

Enter roll number to search: 3

Student Found
Roll No. : 3
Name     : Aarushi
Marks    : 97.00

Students sorted by roll number.

Sorted Student Database:

Roll No.        Name            Marks
---------------------------------------
1               Eshan           92.00
2               Archit          94.00
3               Aarushi         97.00
4               Anushka         97.00
```